

Here's a **line-by-line explanation** of the first three code cells from your uploaded notebook **DL Lab Exp No.2.ipynb**, which is implementing a sentiment analysis model using an LSTM network on the IMDb dataset.

Code Cell 1: Importing Libraries

```
python
CopyEdit
import numpy as np
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Embedding, LSTM, Dropout
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
```

- `numpy`: Used for numerical operations.
 - `tensorflow`: Main deep learning framework.
 - `imdb`: Prebuilt dataset containing movie reviews and sentiment labels (positive/negative).
 - `pad_sequences`: Ensures all review sequences are of equal length.
 - `Sequential`: Used to define a model layer by layer.
 - `Dense`, `Embedding`, `LSTM`, `Dropout`: Common layers in a deep learning model.
 - `Adam`: A popular optimizer.
 - `matplotlib.pyplot`: For plotting training/validation accuracy or loss.
-

Code Cell 2: Load and Preprocess Data

```
python
CopyEdit
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=10000)
max_length = 200
X_train = pad_sequences(X_train, maxlen=max_length)
X_test = pad_sequences(X_test, maxlen=max_length)
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
```

- `imdb.load_data(num_words=10000)`: Loads training and testing data, using only the top 10,000 most frequent words.
 - `max_length = 200`: We fix the maximum review length to 200 words.
 - `pad_sequences`: Ensures all sequences have the same length (pads shorter sequences and truncates longer ones).
 - Prints the shape of padded training and test data: (25000, 200), i.e., 25,000 samples each of 200-word length.
-

Code Cell 3: Build and Compile the Model

```
python
CopyEdit
model = Sequential()
model.add(Embedding(input_dim=10000, output_dim=128))
model.add(LSTM(units=128, dropout=0.2, recurrent_dropout=0.2))
model.add(Dropout(0.5))
model.add(Dense(1, activation='sigmoid'))
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
model.summary()
```

- `Sequential()`: Initializes the model.
 - `Embedding(...)`: Turns integer word indices into dense vectors of size 128.
 - `input_dim=10000`: Vocabulary size.
 - `output_dim=128`: Size of word embeddings.
 - `LSTM(...)`: Long Short-Term Memory layer with 128 units.
 - `dropout=0.2`: Dropout for input connections.
 - `recurrent_dropout=0.2`: Dropout for recurrent connections.
 - `Dropout(0.5)`: Prevents overfitting by dropping 50% of neurons randomly.
 - `Dense(1, activation='sigmoid')`: Output layer for binary classification (positive/negative review).
 - `compile(...)`: Specifies loss function and optimizer.
 - `model.summary()`: Prints the model architecture.
-

Would you like me to continue explaining the next cells from this notebook?

40

You said:

continue for all

ChatGPT said:

Great! Let's continue **line-by-line explanation** for the **remaining code cells** in your notebook **DL Lab Exp No.2.ipynb** — implementing sentiment analysis using LSTM on the IMDB dataset.

Code Cell 4: Train the Model

```
python
CopyEdit
history = model.fit(X_train, y_train, batch_size=64, epochs=5,
validation_data=(X_test, y_test))
```

- `model.fit(...)`: Trains the model on the training data.

- o `X_train, y_train`: Input and target for training.
 - o `batch_size=64`: Processes 64 samples per training step.
 - o `epochs=5`: Runs for 5 complete passes over the training data.
 - o `validation_data=(X_test, y_test)`: Evaluates model on the test data after each epoch.
 - The training process will return a `history` object storing accuracy/loss data.
-

Code Cell 5: Evaluate the Model

```
python
CopyEdit
loss, accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {accuracy * 100:.2f}%")
```

- `model.evaluate(...)`: Evaluates the model on test data.
 - o Returns loss and accuracy.
 - `print(...)`: Displays the test accuracy as a percentage with 2 decimal places.
-

Code Cell 6: Plot Training and Validation Accuracy

```
python
CopyEdit
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

- `history.history[...]`: Contains accuracy values for each epoch.
 - `plt.plot(...)`: Plots training and validation accuracy over epochs.
 - `plt.title(...), plt.xlabel(...), plt.ylabel(...), plt.legend()`: Adds graph details.
 - `plt.show()`: Displays the plot.
-

Code Cell 7: Plot Training and Validation Loss

```
python
CopyEdit
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

- Similar to the previous cell but plotting **loss** instead of accuracy.
 - Helps to observe **overfitting or underfitting** by comparing training and validation loss.
-

☑ Summary of the Notebook:

- Loads and preprocesses the IMDB dataset.
- Builds a deep learning model using **Embedding + LSTM** layers.
- Trains the model for sentiment classification.
- Evaluates and visualizes model performance.

Let's continue with a **line-by-line explanation** for your file **DL Lab Exp No.3.ipynb** — which implements **image classification using a CNN on the CIFAR-10 dataset**.

☑ Code Cell 1: Importing Libraries

```
python
CopyEdit
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical
```

- `numpy`: For numerical operations.
 - `matplotlib.pyplot`: To plot images and graphs.
 - `cifar10`: CIFAR-10 dataset (60,000 32x32 color images in 10 classes).
 - `Sequential`: Linear stack of layers to build the CNN.
 - **CNN layers**:
 - `Conv2D`: Convolution operation.
 - `MaxPooling2D`: Downsampling.
 - `Flatten`: Converts 2D feature maps to 1D.
 - `Dense`: Fully connected layers.
 - `Dropout`: Prevents overfitting by randomly dropping neurons.
 - `to_categorical`: Converts class labels to one-hot encoded format.
-

☑ Code Cell 2: Load and Preprocess the CIFAR-10 Dataset

```
python
CopyEdit
(X_train, y_train), (X_test, y_test) = cifar10.load_data()
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0
```

```
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

- `cifar10.load_data()`: Loads training and test sets.
 - `X_train.astype('float32') / 255.0`: Normalizes pixel values to [0, 1].
 - `to_categorical(..., 10)`: Converts integer class labels to 10-dimensional one-hot vectors.
-

☒ Code Cell 3: Build the CNN Model

```
python
CopyEdit
model = Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(MaxPooling2D((2, 2)))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D((2, 2)))
model.add(Flatten())
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(10, activation='softmax'))
```

- `Conv2D(32, (3, 3), ...)`: 32 filters of size 3x3 applied to input image.
 - `MaxPooling2D((2, 2))`: Reduces feature map size by half.
 - `Conv2D(64, ...)`: Adds another convolutional layer with 64 filters.
 - `Flatten()`: Converts 3D output to 1D for fully connected layers.
 - `Dense(64, ...)`: Fully connected layer with 64 units and ReLU activation.
 - `Dropout(0.5)`: 50% neurons randomly dropped.
 - `Dense(10, softmax)`: Output layer for 10 classes with softmax to get class probabilities.
-

☒ Code Cell 4: Compile the Model

```
python
CopyEdit
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
```

- `optimizer='adam'`: Optimizer for updating weights.
 - `loss='categorical_crossentropy'`: Suitable for multi-class classification.
 - `metrics=['accuracy']`: Monitors accuracy during training and evaluation.
-

☒ Code Cell 5: Train the Model

```
python
CopyEdit
```

```
history = model.fit(X_train, y_train, batch_size=64, epochs=10,  
validation_data=(X_test, y_test))
```

- Trains the model with:
 - Batch size of 64 images.
 - 10 epochs.
 - Validation on test data to monitor overfitting.
 - Returns `history` containing training/validation accuracy and loss.
-

☑ Code Cell 6: Evaluate the Model

```
python  
CopyEdit  
loss, accuracy = model.evaluate(X_test, y_test)  
print(f"Test Accuracy: {accuracy * 100:.2f}%")
```

- Evaluates model performance on test data.
 - Prints accuracy percentage.
-

☑ Code Cell 7: Plot Accuracy

```
python  
CopyEdit  
plt.plot(history.history['accuracy'], label='Train Accuracy')  
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')  
plt.title('Model Accuracy')  
plt.xlabel('Epoch')  
plt.ylabel('Accuracy')  
plt.legend()  
plt.show()
```

- Plots training and validation accuracy per epoch using matplotlib.
-

☑ Code Cell 8: Plot Loss

```
python  
CopyEdit  
plt.plot(history.history['loss'], label='Train Loss')  
plt.plot(history.history['val_loss'], label='Validation Loss')  
plt.title('Model Loss')  
plt.xlabel('Epoch')  
plt.ylabel('Loss')  
plt.legend()  
plt.show()
```

- Same as above, but for loss — helps visualize model convergence and overfitting.

 Summary:

- Loaded and normalized CIFAR-10 data.
- Built a CNN with convolution, pooling, dense, and dropout layers.
- Trained the model and evaluated performance.
- Visualized results using plots.